

Program Analysis 2025-26

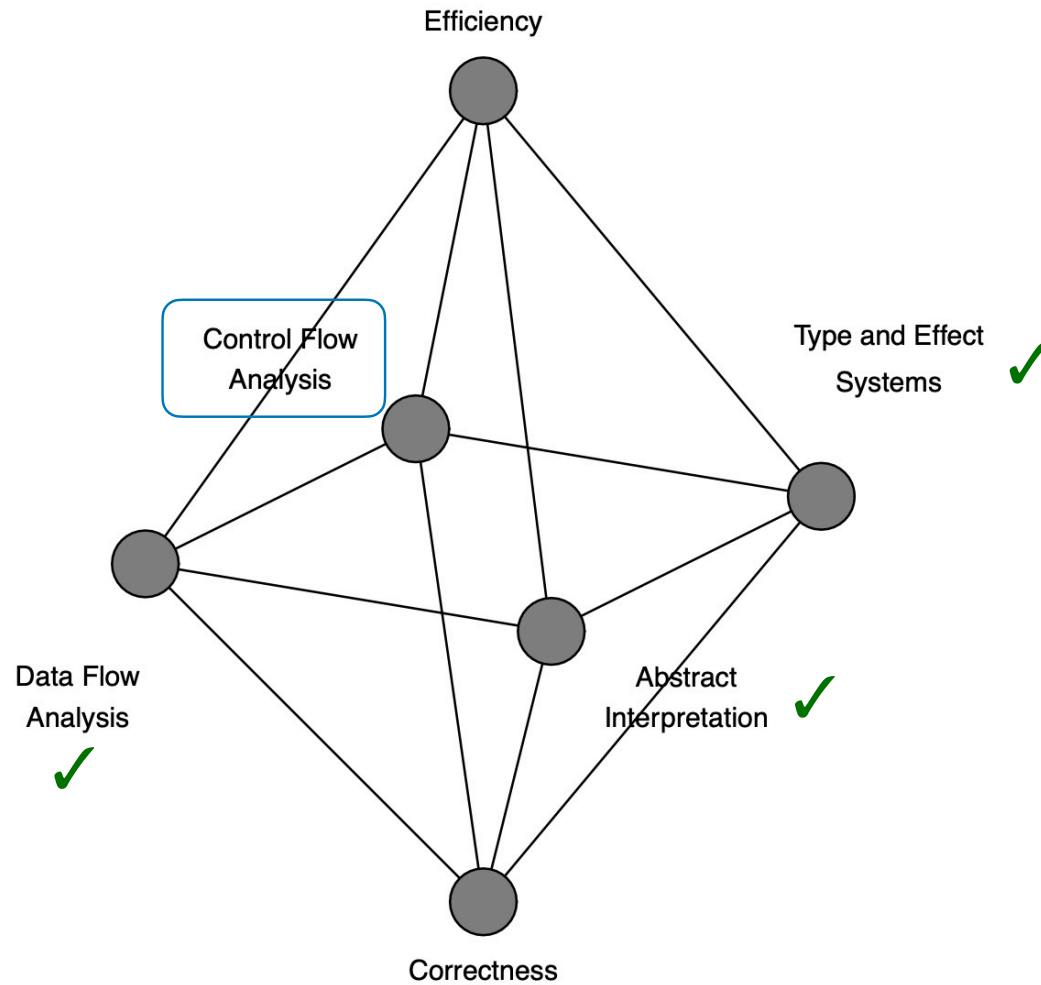
Lecture #23

Control Flow Analysis



UNIVERSITÀ
DI PISA

Program Analysis



Control Flow Analysis

Flow information is essential for the specification of Data Flow Analyses (e.g., Monotone Framework over Control Flow Graphs)

Simple imperative languages: flow information can be extracted directly from the program text: procedure calls are performed by explicitly mentioning the name of a procedure

But what if functions are values?

Now we have a problem: we often do not know at compile time which function is called

Control Flow Analysis (CFA) statically approximates which functions **may** be called at each call site

Control Flow Analysis

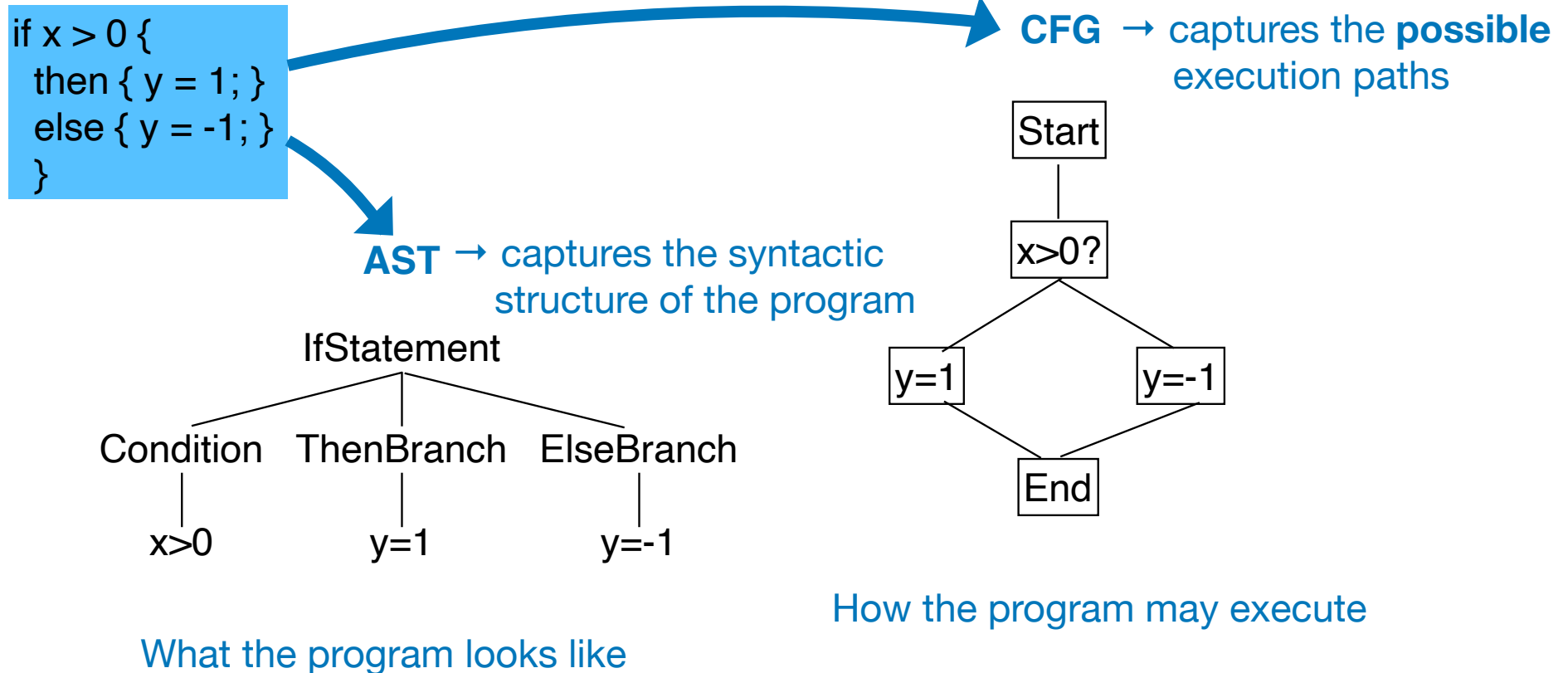
Control flow describes the rules underlying the evolution of the program

- implicit in the program code and in the **Abstract Syntax Tree (AST)**
- explicit in **Control Flow Graph (CFG)**, which captures the order of execution and possible paths (assuming the targets of calls are known)

The purpose of **Control Flow Analysis (CFA)** is to determine which blocks may lead to which other blocks

But in modern languages, even building the CFG is not trivial

AST and CFG



To build the CFG, we must know how control may flow at runtime

Analyzing CFGs

So far looked in previous
courses

- **intraprocedural** analysis: → analyze a single function
- **interprocedural** analysis: → analyze the whole program with function calls

Until now, all targets were known statically

Interprocedural analysis

- Construct a CFG for each function
- Glue them together via **function calls** and **returns**

Interprocedural analysis

Interprocedural analysis:

tractable when the program has a **simple structure**

→ calls have **statically known targets**

Simple imperative language with higher-order functions

Higher-order functions complicate CFG construction:

- several functions may be invoked at a call site
- this depends on the dataflow
- but dataflow analysis first requires a CFG

Simple imperative language with higher-order functions

```
inc(i) { return i+1; }  
dec(j) { return j-1; }  
ide(k) { return k; }
```

```
foo(n,f) {  
  var r;  
  if (n==0) { f=ide; }  
  r = f(n);  
  return r;  
}
```

What is f?

```
main() {  
  var x,y;  
  x = input;  
  if (x>0) { y = foo(x,inc); } else { y = foo(x,dec); }  
  return y;  
}
```

Example by A. Møller & M. I. Schwartzbach

A **control flow analysis**
approximates the call graph

$\llbracket f \rrbracket = \{\text{inc}, \text{dec}, \text{ide}\}$

Functional languages

Control-flow (CFG) + data “flowing” through it

→ no longer works

Why?

- The boundary between control and data becomes unclear
- Everything is both data and control

Functions are higher-order: they can be at the same time:

- code to be executed and
- data, i.e., arguments or results of other function

No clear separation between control-flow and data-flow

When the program decides “at runtime” which function to call, the control flow is no longer explicit

Dynamic Dispatch Problem

Consider the following program fragment

```
let
  f = fn x => x 1;
  g = fn y => y + 2;
  h = fn z => z + 3
in
  (f g) + (f h)
```

variables can denote functions and
which procedure/function gets invoked
depends on runtime values

- x is a function variable
- at runtime: $x \in \{g, h\}$

So call $x\ 1$ may target g or h (from calls $(f\ g)$ and $(f\ h)$)

Here it is clear which functions are actually called, but in general it is not

Dynamic Dispatch Problem

Consider the following program fragment

```
let
  f = fn x => x 1;
  g = fn y => y + 2;
  h = fn z => z + 3
in
  (f g) + (f h)
```

We cannot know the exact target of a call

So we compute a **safe approximation**

Solution: static analysis

→ **over-approximate** the **possible targets** of each call

Control Flow Analysis (CFA):

→ For each function application (call site), compute a **safe approximation**

How? By solving a system of constraints, where constraints describe how values flow between variables

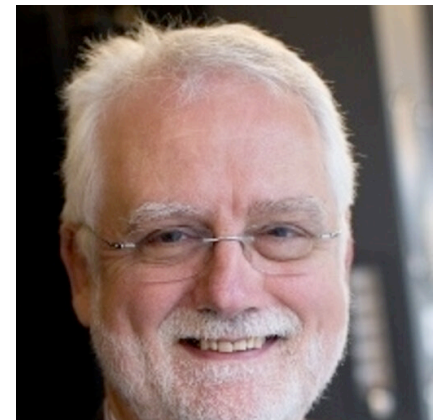
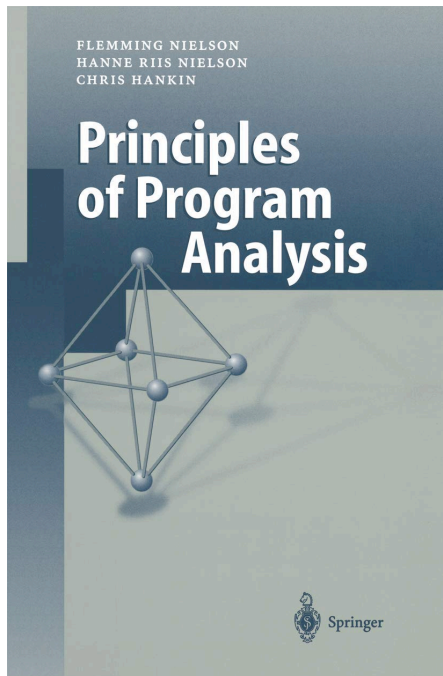
Control Flow Analysis

Control Flow Analysis: historically

- Reynolds: early analyses of LISP programs
 - Jones: first to study lambda expressions → introduced control-flow analysis
 - Shivers: formalized CFA for Scheme → basis for constraint-based approaches
 - ...
-
- Today: constraint-based CFA for higher-order programs

The Constraint Based Approach

We adopt the approach illustrated in *Principles of Program Analysis* by Flemming Nielson, Hanne Riis Nielson, Chris Hankin **[Chapter 3]**



FUN: syntax

term in **Term**

$t ::= c$ // constant in **Const**

| x // variable in **Var**

| $\text{fn } x \Rightarrow e_0$ // function definition (abstraction)

| $\text{fun } f \ x \Rightarrow e_0$ // recursive function definition

| $e_1 \ e_2$ // function application

| $\text{if } e_0 \text{ then } e_1 \text{ else } e_2$ // conditional

| $\text{let } x = e_1 \text{ in } e_2$ // local definition

| $e_1 \ \text{op} \ e_2$ // binary operation (op in **Op**)

expression
(labelled term)
in **Exp**

$e ::= t^\ell$

Label:
needed for the CFA
identify program
points (e.g., call sites)

Each label ℓ in **Lab** corresponds to a program point where CFA computes information

CFA of FUN

The goal

Associate for each variable x and each expression e , a **safe over-approximation** of the set of functions that they may evaluate to at runtime

valid

This determines the possible targets of function applications

Example of function application

```
( if e then ((*) 2) else ((+) 1) ) 10
```

- e evaluates to either $((*) 2)$ or $((+) 1)$
- the call may target $\{(*), (+)\}$

Example 1: labelling

Consider the following expression and label it

`((fn x => x )(fn y => y ))`

`((fn x => x1)2 (fn y => y3)4)5`

`(fn x => x1)2`

`(fn y => y3)4`

`x` `x1`

`y` `y3`

Example 1

Consider the following expression

$$((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$$

Let us look at what happens at runtime: $((\lambda x . x)(\lambda y . y)) \rightarrow x\{(\lambda y . y)/x\} \equiv (\lambda y . y)$

- x is bound to $(\text{fn } y \Rightarrow y^3)$
- the subexpression at label 1 evaluates to that function
- the overall expression at label 5 evaluates to that function

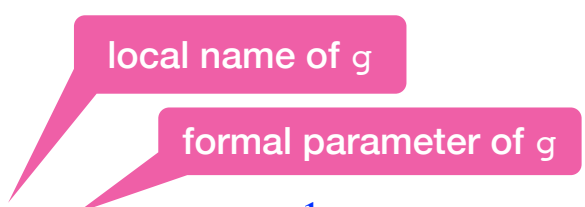
in λ -calculus

CFA will compute a **safe approximation** of these results

Example 2

Consider the following expression

$\text{loop} \triangleq (\text{let } g = (\text{fun } f \ x \Rightarrow (f^1 \ (\text{fn } y \Rightarrow y^2)^3)^4)^5$
 $\text{in } (g^6 \ (\text{fn } z \Rightarrow z^7)^8)^9)^{10}$



g is a **recursive function** applied to the identity function $(\text{fn } z \Rightarrow z^7)$ that

- ignores its actual parameter and
- calls itself recursively with the argument $(\text{fn } y \Rightarrow y^2)$
- so the program loops

$\Rightarrow$ possible recursive flow of function values:

CFA must **approximate this recursive behavior**

The Control Flow Analysis

CFA as a Constraint Based Analysis

- A 0-CFA is a **Constraint Based Analysis**
- it is defined via **inference rules that generate constraints** over the possible dataflow values for each variable or labeled location; those
- constraints are then solved

CFA with Flow Logic

Flow Logic is a declarative approach to program analysis that mainly separates the **specification** from the actual **computation**

- **specification**: describes when the analysis results are acceptable
- **computation**: solving the constraints derived by the specification

acceptability
relation

constraint
satisfaction
problem

CFA: overview

We define our CFA in several steps:

- **Specification:** define which sets of functions are acceptable (acceptability relation)
- **Syntax-directed rules:** refine the acceptability relation to be syntax directed
- **Constraint generation:** turn the specification into a finite set of constraints
- **Constraint solving:** compute the least solution of the constraints (the analysis result)

Abstract domains: informally

the simplest
form of CFA

The result of a **0-CFA** analysis is a pair $(\hat{C}, \hat{\rho})$:

- $\hat{C}: \ell \mapsto$ set of function abstractions the subexpression labelled ℓ **may** evaluate to
- $\hat{\rho}: x \mapsto$ set of function abstractions, x **may** be bound to

These are abstract values

Goal:

For each $(t_1^{\ell_1} t_2^{\ell_2})^{\ell}$ determine the possible functions for t_1 from the set of function abstractions t_2 **may** evaluate to

0-CFA means that **no context information** is considered, e.g., same function set at all call sites

Abstract domains: formally

The result of a CFA analysis is a pair $(\hat{C}, \hat{\rho})$:

$$\hat{C} \in \widehat{\text{Cache}} : \text{Lab} \rightarrow \widehat{\text{Val}}$$

$$\hat{\rho} \in \widehat{\text{Env}} : \text{Var} \rightarrow \widehat{\text{Val}}$$

$$\hat{v} \in \widehat{\text{Val}} = \wp(\text{Term})$$

Only functions,
no constants
(no DFA component):
pure CFA

$\text{fn } x \Rightarrow e_0$ abstracts the
corresponding function
in all contexts

- An **abstract value** abstracts a **set of functions**:,
i.e., terms in the form $\text{fn } x \Rightarrow e_0$ or $\text{fun } f \ x \Rightarrow e_0$
- An **abstract cache** $\hat{C}$ associates abstract values
with each labelled program point
- An **abstract environment** $\hat{\rho}$ associates abstract
values with each variable

We assume that all variables and labels are distinct

Back to Example 1

Consider the following 0-CFA guess

$$((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$$

	$(\hat{C}_e, \hat{\rho}_e)$
$\hat{C}_e$	1 {fn y => y ³ }
	2 {fn x => x ¹ }
	3 $\emptyset$
	4 {fn y => y ³ }
	5 {fn y => y ³ }
$\hat{\rho}_e$	x {fn y => y ³ }
	y $\emptyset$

We want to know which functions **may** be bound to **variables** or **labels** (terms)

The variable y is never evaluated

Does $(\hat{C}_e, \hat{\rho}_e)$ safely over-approximate the runtime behavior? Is it an acceptable guess? 

Back to Example 1

Consider also the following 0-CFA guesses

$$((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$$

	$(\hat{C}_e, \hat{\rho}_e)$	$(\hat{C}'_e, \hat{\rho}'_e)$	$(\hat{C}''_e, \hat{\rho}''_e)$
1	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^3\}$
2	$\{\text{fn } x \Rightarrow x^1\}$	$\{\text{fn } x \Rightarrow x^1\}$	$\{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^3\}$
3	$\emptyset$	$\emptyset$	$\{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^3\}$
4	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^3\}$
5	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^3\}$
x	$\{\text{fn } y \Rightarrow y^3\}$	$\emptyset$	$\{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^3\}$
y	$\emptyset$	$\emptyset$	$\{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^3\}$





X

false
negative



Are they acceptable? Conservative (over-approximating) guesses are acceptable, incomplete ones are not

The specification of a solution

Given a guess $(\hat{C}, \hat{\rho})$, we determine whether it is an **acceptable** CFA

A guess is acceptable if it safely over-approximates the runtime behavior

$$(\hat{C}, \hat{\rho}) \models e$$

relation

$$\models: \widehat{\mathbf{Cache}} \times \widehat{\mathbf{Env}} \times \mathbf{Exp} \rightarrow \{\text{true}, \text{false}\}$$

It means that the guess is acceptable for expression e

Correct prediction

- If t^ℓ evaluates to a function, then it must be included in $\hat{C}(\ell)$
- If x is bound to a function, then it must be included in $\hat{\rho}(x)$

Acceptability relation

$$(\hat{C}, \hat{\rho}) \models e$$

To define acceptability, we specify correctness rules for each syntactic category in FUN

Each rule ensures that the guess includes all possible flows

Clause for constants

$$[con] (\hat{C}, \hat{\rho}) \models c^\ell \text{ always}$$

that can be reformulated as

$$[con] (\hat{C}, \hat{\rho}) \models c^\ell \text{ iff } \emptyset \subseteq \hat{C}(\ell)$$

No demands on $\hat{C}(\ell)$: constants do not evaluate to functions
In pure 0-CFA, we do **not** track data values

(no functions
can flow here)

Example

$$e \triangleq 7^1$$

$$\hat{C}(1) = \emptyset$$

Clause for variables

$$[var] (\hat{C}, \hat{\rho}) \models x^\ell \text{ iff } \hat{\rho}(x) \subseteq \hat{C}(\ell)$$

Everything the variable x can be evaluated to must be included in what is observed at program point ℓ

This propagates information from variables to program points: the flow goes from $\hat{\rho}$ to $\hat{C}$ (we get the value from the variable into the program)

Example

$$e = x^1$$

$$\hat{C}(1) \supseteq \hat{\rho}(x) \supseteq \{(\text{fn } y \Rightarrow y^3)\} \text{ [See example 1]}$$

Clause for functions

$$[fn] (\hat{C}, \hat{\rho}) \models (fn\ x \Rightarrow e_0)^\ell \text{ iff } \{fn\ x \Rightarrow e_0\} \subseteq \hat{C}(\ell)$$

functions are
abstract
values

The functional term $fn\ x \Rightarrow e_0$ must be included in $\hat{C}(\ell)$

This means that it may occur as part of a **closure** at program point ℓ during evaluation

No checks on the body e_0 : the clause for application will handle the body

Example

$$e \triangleq (fn\ x \Rightarrow x^1)^2$$

$$\hat{C}(2) \supseteq \{(fn\ x \Rightarrow x^1)\} \dots$$

Clause for recursive functions

$$[fun] (\hat{C}, \hat{\rho}) \models (\text{fun } f \ x \Rightarrow e_0)^\ell \text{ iff } \{\text{fun } f \ x \Rightarrow e_0\} \subseteq \hat{C}(\ell)$$

functions are
abstract
values

The functional term $\text{fn } x \Rightarrow e_0$ must be included in $\hat{C}(\ell)$

This means that it may occur as part of a **closure** at program point ℓ during evaluation

No checks on the body e_0 : the clause for application will handle the body

Example

$$e \triangleq (\text{fun } f \ x \Rightarrow (f^1 (\text{fn } y \Rightarrow y^2)^3)^4)^5$$
$$\hat{C}(5) \supseteq \{\text{fun } f \ x \Rightarrow (f^1 (\text{fn } y \Rightarrow y^2)^3)^4\} \dots$$

Clause for conditional

$$\begin{aligned} [if] \ (\hat{C}, \hat{\rho}) \models (\text{if } t_0^{\ell_0} \text{ then } t_1^{\ell_1} \text{ else } t_2^{\ell_2})^\ell \text{ iff } & (\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \\ & \wedge (\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge \hat{C}(\ell_1) \subseteq \hat{C}(\ell) \\ & \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2} \wedge \hat{C}(\ell_2) \subseteq \hat{C}(\ell) \end{aligned}$$

Clause for conditional

$$[if] (\hat{C}, \hat{\rho}) \models (\text{if } t_0^{\ell_0} \text{ then } t_1^{\ell_1} \text{ else } t_2^{\ell_2})^\ell \text{ iff } (\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge (\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge \hat{C}(\ell_1) \subseteq \hat{C}(\ell) \\ \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2} \wedge \hat{C}(\ell_2) \subseteq \hat{C}(\ell)$$

This clause

- requires checking the sub-expressions:
hence it uses “recursive calls” $(\hat{C}, \hat{\rho}) \models t_i^{\ell_i}$
- propagates the values produced by sub-expression $\hat{C}(\ell_i)$ to the overall expression $\hat{C}(\ell)$

Condition is ignored:
over-approximation

This corresponds to merging the possible values from **both** branches

Example

$e \triangleq (\text{if } (x^1 > 0^2)^3 \text{ then } (\text{fn } y \Rightarrow y^4)^5 \text{ else } (\text{fn } z \Rightarrow z^6)^7)^8$

$\hat{C}(5) \subseteq \hat{C}(8)$ and $\hat{C}(7) \subseteq \hat{C}(8) \dots$

Clause for binary operation

$$[op] (\hat{C}, \hat{\rho}) \models (t_1^{\ell_1} \text{ op } t_2^{\ell_2})^\ell \text{ iff } (\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2}$$

This clause simply

- requires checking the **sub-expressions**: hence it uses “recursive calls”

$$(\hat{C}, \hat{\rho}) \models t_i^{\ell_i}$$

No demands on $\hat{C}(\ell)$

In pure 0-CFA, we do **not** track data values

Example

$$e \triangleq (7^1 + 4^2)^3$$

$$\hat{C}(3) = \emptyset \dots$$

Clause for the let binding

$$[let] (\hat{C}, \hat{\rho}) \models (\text{let } x = t_1^{\ell_1} \text{ in } t_2^{\ell_2})^\ell \text{ iff } (\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2} \wedge \\ \hat{C}(\ell_1) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_2) \subseteq \hat{C}(\ell)$$

Clause for the let binding

$[let] (\hat{C}, \hat{\rho}) \models (\text{let } x = t_1^{\ell_1} \text{ in } t_2^{\ell_2})^\ell \text{ iff } (\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2} \wedge$

$$\hat{C}(\ell_1) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_2) \subseteq \hat{C}(\ell)$$

This clause

- requires checking the **sub-expressions**:
hence it uses “recursive calls” $(\hat{C}, \hat{\rho}) \models t_i^{\ell_i}$

the value of x must be used when analysing t_2

Double flow: propagation of the values

- from $\hat{C}(\ell_1)$ to $\hat{\rho}(x)$ [**binding**]
- from $\hat{C}(\ell_2)$ to $\hat{C}(\ell)$ [**evaluation**]

Example

$e \triangleq (\text{let } f = (\text{fn } x \Rightarrow x^1)^2 \text{ in } (f^3 3^4)^5)^6$

$\hat{C}(2) \subseteq \hat{\rho}(x)$ and $\hat{C}(5) \subseteq \hat{C}(6) \dots$

Clause for function application

$[app] (\hat{C}, \hat{\rho}) \models (t_1^{\ell_1} t_2^{\ell_2})^\ell$ iff $(\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2} \wedge$

$(\forall (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) :$
 $(\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge$

$\hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge$

$(\forall (\text{fun } f x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) :$
 $(\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge$

$\hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge$
 $\{\text{fun } f x \Rightarrow e_0\} \subseteq \hat{\rho}(f))$



KEY CLAUSE
 It simulates all possible function calls

Clause for function application

$$[app] (\hat{C}, \hat{\rho}) \models (t_1^{\ell_1} t_2^{\ell_2})^{\ell} \text{ iff } (\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2} \wedge$$

This clause requires checking the **sub-expressions**: hence it uses “recursive calls”

$$(\hat{C}, \hat{\rho}) \models t_i^{\ell_i}$$

Clause for function application

$$[app] (\hat{C}, \hat{\rho}) \models (t_1^{\ell_1} t_2^{\ell_2})^{\ell} \text{ iff}$$

For each term $(\text{fn } x \Rightarrow t_0^{\ell_0})$ that may reach ℓ_1 it requires that

- the body can be analysed
- the actual parameter (with label ℓ_2) is linked to the formal parameter x
- the result of the function evaluation (with label ℓ_0) is linked to the result of the application itself (with label ℓ)

for each function
the can be applied

$$(\forall (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) : \\ (\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge \\ \hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge$$

the argument “flows
into” the variable

What comes out the
evaluation of the body
comes out also from the
application

Clause for function application

$[app] (\hat{C}, \hat{\rho}) \models (t_1^{\ell_1} t_2^{\ell_2})^\ell$ iff

For each term $(\text{fun } f \ x \Rightarrow t_0^{\ell_0})$
that may reach ℓ_1 it requires that

- ... same as before and
- the term must be included in $\hat{\rho}(f)$ to reflect the recursive nature of the construct

$$\begin{aligned}
 & (\forall (\text{fun } f \ x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) : \\
 & \quad (\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge \\
 & \quad \hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell) \wedge \\
 & \quad \{\text{fun } f \ x \Rightarrow e_0\} \subseteq \hat{\rho}(f))
 \end{aligned}$$

Clause for function application

$$[app] (\hat{C}, \hat{\rho}) \models (t_1^{\ell_1} t_2^{\ell_2})^\ell \text{ iff } (\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2}$$

Application = simulate all possible calls

For each function in $\hat{C}(\ell_1)$ (the function position):

- analyze body ($t_0^{\ell_0}$)
- bind argument (via $\hat{\rho}(x)$)
- propagate results

Results are merged in $\hat{C}(\ell)$

$$(\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2}$$

$$(\forall (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) :$$

$$(\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge$$

$$\hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge$$

$$(\forall (\text{fun } f \ x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) :$$

$$(\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge$$

$$\hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge$$

$$\{\text{fun } f \ x \Rightarrow e_0\} \subseteq \hat{\rho}(f))$$

Clause for function application (recap)

Application clause

- combines function and argument
- drives propagation

See the more complete **Example 1** next

Back to Example 1

$(\hat{C}_e, \hat{\rho}_e)$ is an **acceptable** guess for $((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$

$(\hat{C}_e, \hat{\rho}_e) \models ((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$ iff

	$(\hat{C}_e, \hat{\rho}_e)$
1	$\{\text{fn } y \Rightarrow y^3\}$
2	$\{\text{fn } x \Rightarrow x^1\}$
3	$\emptyset$
4	$\{\text{fn } y \Rightarrow y^3\}$
5	$\{\text{fn } y \Rightarrow y^3\}$
x	$\{\text{fn } y \Rightarrow y^3\}$
y	$\emptyset$

$$\left\{ \begin{array}{l} (\hat{C}_e, \hat{\rho}_e) \models (\text{fn } x \Rightarrow x^1)^2 \text{ iff } \{\text{fn } x \Rightarrow x^1\} \subseteq \hat{C}_e(2) \\ (\hat{C}_e, \hat{\rho}_e) \models (\text{fn } y \Rightarrow y^3)^4 \text{ iff } \{\text{fn } y \Rightarrow y^3\} \subseteq \hat{C}_e(4) \end{array} \right.$$

Since $(\text{fn } x \Rightarrow x^1) = \hat{C}_e(2)$ so we must also check

$$\left\{ \begin{array}{l} (\hat{C}_e, \hat{\rho}_e) \models x^1 \text{ iff } \hat{\rho}(x) \subseteq \hat{C}_e(1) \\ \hat{C}_e(4) \subseteq \hat{\rho}(x) \\ \hat{C}_e(1) \subseteq \hat{C}_e(5) \end{array} \right.$$

$$\begin{aligned} [app] \ (\tilde{C}, \hat{\rho}) \models (t_1^{\ell_1} t_2^{\ell_2})^\ell \text{ iff } & (\tilde{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge (\tilde{C}, \hat{\rho}) \models t_2^{\ell_2} \\ & (\forall (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) : \\ & \quad (\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge \\ & \quad \hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell) \wedge \end{aligned}$$

The analysis generates constraints

Back to Example 1

$(\hat{C}_e, \hat{\rho}_e)$ is an **acceptable** guess for $((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$

	$(\hat{C}_e, \hat{\rho}_e)$		
1	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } x \Rightarrow x^1\} \subseteq \hat{C}_e(2)$	✓
2	$\{\text{fn } x \Rightarrow x^1\}$	$\{\text{fn } y \Rightarrow y^3\} \subseteq \hat{C}_e(4)$	✓
3	$\emptyset$		
4	$\{\text{fn } y \Rightarrow y^3\}$	$\hat{\rho}(x) \subseteq \hat{C}_e(1)$	✓
5	$\{\text{fn } y \Rightarrow y^3\}$	$\hat{C}_e(4) \subseteq \hat{\rho}(x)$	✓
x	$\{\text{fn } y \Rightarrow y^3\}$	$\hat{C}_e(1) \subseteq \hat{C}_e(5)$	✓
y	$\emptyset$		



Back to Example 1

$(\hat{C}'_e, \hat{\rho}'_e)$ is **non acceptable** for $((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$

$(\hat{C}'_e, \hat{\rho}'_e) \models ((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$ iff

	$(\hat{C}_e, \hat{\rho}_e)$	$(\hat{C}'_e, \hat{\rho}'_e)$
1	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$
2	$\{\text{fn } x \Rightarrow x^1\}$	$\{\text{fn } x \Rightarrow x^1\}$
3	$\emptyset$	$\emptyset$
4	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$
5	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$
x	$\{\text{fn } y \Rightarrow y^3\}$	$\emptyset$
y	$\emptyset$	$\emptyset$

$(\hat{C}'_e, \hat{\rho}'_e) \models (\text{fn } x \Rightarrow x^1)^2$ iff $\{\text{fn } x \Rightarrow x^1\} \subseteq \hat{C}'_e(2)$

$(\hat{C}'_e, \hat{\rho}'_e) \models (\text{fn } y \Rightarrow y^3)^4$ iff $\{\text{fn } y \Rightarrow y^3\} \subseteq \hat{C}'_e(4)$

Since $(\text{fn } x \Rightarrow x^1) = \hat{C}_e(e)$ so we must also check

$(\hat{C}'_e, \hat{\rho}'_e) \models x^1$ iff $\hat{\rho}'_e(x) \subseteq \hat{C}'_e(1)$

$\hat{C}'_e(4) \subseteq \hat{\rho}'_e(x)$ **X**

$\hat{C}'_e(1) \subseteq \hat{C}'_e(5)$

missing possible
value of x

A guess is **acceptable** only if all constraints hold

Back to Example 1

$(\hat{C}_e, \hat{\rho}_e)$ is an **acceptable** guess for $((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$

Intuitively, we solve the constraints iteratively until reaching a fix point

$$\{\text{fn } x \Rightarrow x^1\} \subseteq \hat{C}_e(2)$$

$$\{\text{fn } y \Rightarrow y^3\} \subseteq \hat{C}_e(4)$$

$$\hat{\rho}(x) \subseteq \hat{C}_e(1)$$

$$\hat{C}_e(4) \subseteq \hat{\rho}(x)$$

$$\hat{C}_e(1) \subseteq \hat{C}_e(5)$$

$$\hat{C}_e(1)$$

$$\hat{C}_e(2)$$

$$\hat{C}_e(3)$$

$$\hat{C}_e(4)$$

$$\hat{C}_e(5)$$

$$\hat{\rho}(x)$$

$$\hat{\rho}(y)$$

Back to Example 1

$((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$

$\{\text{fn } x \Rightarrow x^1\} \subseteq \hat{C}_e(2)$

$\{\text{fn } y \Rightarrow y^3\} \subseteq \hat{C}_e(4)$

$\hat{\rho}(x) \subseteq \hat{C}_e(1)$

$\hat{C}_e(4) \subseteq \hat{\rho}(x)$

$\hat{C}_e(1) \subseteq \hat{C}_e(5)$

$\hat{C}_e(1)$	$\emptyset$	
$\hat{C}_e(2)$	$\emptyset$	
$\hat{C}_e(3)$	$\emptyset$	
$\hat{C}_e(4)$	$\emptyset$	
$\hat{C}_e(5)$	$\emptyset$	
$\hat{\rho}(x)$	$\emptyset$	
$\hat{\rho}(y)$	$\emptyset$	

Back to Example 1

$((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$

$$\{\text{fn } x \Rightarrow x^1\} \subseteq \hat{C}_e(2)$$

$$\{\text{fn } y \Rightarrow y^3\} \subseteq \hat{C}_e(4)$$

$$\hat{\rho}(x) \subseteq \hat{C}_e(1)$$

$$\hat{C}_e(4) \subseteq \hat{\rho}(x)$$

$$\hat{C}_e(1) \subseteq \hat{C}_e(5)$$

$\hat{C}_e(1)$	$\emptyset$	$\{\text{fn } x \Rightarrow x^1\}$	
$\hat{C}_e(2)$	$\emptyset$	$\emptyset$	
$\hat{C}_e(3)$	$\emptyset$	$\emptyset$	
$\hat{C}_e(4)$	$\emptyset$	$\{\text{fn } y \Rightarrow y^3\}$	
$\hat{C}_e(5)$	$\emptyset$	$\emptyset$	
$\hat{\rho}(x)$	$\emptyset$	$\emptyset$	
$\hat{\rho}(y)$	$\emptyset$	$\emptyset$	

Back to Example 1

$$((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$$

	$(\hat{C}_e, \hat{\rho}_e)$			
1	$\{\text{fn } y \Rightarrow y^3\}$	$\hat{C}_e(1)$	$\emptyset$	$\{\text{fn } x \Rightarrow x^1\}$
2	$\{\text{fn } x \Rightarrow x^1\}$	$\hat{C}_e(2)$	$\emptyset$	$\{\text{fn } x \Rightarrow x^1\}$
3	$\emptyset$	$\hat{C}_e(3)$	$\emptyset$	$\emptyset$
4	$\{\text{fn } y \Rightarrow y^3\}$	$\hat{C}_e(4)$	$\emptyset$	$\{\text{fn } y \Rightarrow y^3\}$
5	$\{\text{fn } y \Rightarrow y^3\}$	$\hat{C}_e(5)$	$\emptyset$	$\{\text{fn } x \Rightarrow x^1\}$
x	$\{\text{fn } y \Rightarrow y^3\}$	$\hat{\rho}(x)$	$\emptyset$	$\{\text{fn } y \Rightarrow y^3\}$
y	$\emptyset$	$\hat{\rho}(y)$	$\emptyset$	$\emptyset$

CFA

- [con] $(\widehat{C}, \widehat{\rho}) \models c^\ell$ always
- [var] $(\widehat{C}, \widehat{\rho}) \models x^\ell$ iff $\widehat{\rho}(x) \subseteq \widehat{C}(\ell)$
- [fn] $(\widehat{C}, \widehat{\rho}) \models (\mathbf{fn} \ x \Rightarrow e_0)^\ell$ iff $\{\mathbf{fn} \ x \Rightarrow e_0\} \subseteq \widehat{C}(\ell)$
- [fun] $(\widehat{C}, \widehat{\rho}) \models (\mathbf{fun} \ f \ x \Rightarrow e_0)^\ell$ iff $\{\mathbf{fun} \ f \ x \Rightarrow e_0\} \subseteq \widehat{C}(\ell)$
- [app] $(\widehat{C}, \widehat{\rho}) \models (t_1^{\ell_1} \ t_2^{\ell_2})^\ell$
iff $(\widehat{C}, \widehat{\rho}) \models t_1^{\ell_1} \wedge (\widehat{C}, \widehat{\rho}) \models t_2^{\ell_2} \wedge$
 $(\forall (\mathbf{fn} \ x \Rightarrow t_0^{\ell_0}) \in \widehat{C}(\ell_1) :$
 $(\widehat{C}, \widehat{\rho}) \models t_0^{\ell_0} \wedge$
 $\widehat{C}(\ell_2) \subseteq \widehat{\rho}(x) \wedge \widehat{C}(\ell_0) \subseteq \widehat{C}(\ell)) \wedge$
 $(\forall (\mathbf{fun} \ f \ x \Rightarrow t_0^{\ell_0}) \in \widehat{C}(\ell_1) :$
 $(\widehat{C}, \widehat{\rho}) \models t_0^{\ell_0} \wedge$
 $\widehat{C}(\ell_2) \subseteq \widehat{\rho}(x) \wedge \widehat{C}(\ell_0) \subseteq \widehat{C}(\ell) \wedge$
 $\{\mathbf{fun} \ f \ x \Rightarrow t_0^{\ell_0}\} \subseteq \widehat{\rho}(f))$
- [if] $(\widehat{C}, \widehat{\rho}) \models (\mathbf{if} \ t_0^{\ell_0} \ \mathbf{then} \ t_1^{\ell_1} \ \mathbf{else} \ t_2^{\ell_2})^\ell$
iff $(\widehat{C}, \widehat{\rho}) \models t_0^{\ell_0} \wedge$
 $(\widehat{C}, \widehat{\rho}) \models t_1^{\ell_1} \wedge (\widehat{C}, \widehat{\rho}) \models t_2^{\ell_2} \wedge$
 $\widehat{C}(\ell_1) \subseteq \widehat{C}(\ell) \wedge \widehat{C}(\ell_2) \subseteq \widehat{C}(\ell)$
- [let] $(\widehat{C}, \widehat{\rho}) \models (\mathbf{let} \ x = t_1^{\ell_1} \ \mathbf{in} \ t_2^{\ell_2})^\ell$
iff $(\widehat{C}, \widehat{\rho}) \models t_1^{\ell_1} \wedge (\widehat{C}, \widehat{\rho}) \models t_2^{\ell_2} \wedge$
 $\widehat{C}(\ell_1) \subseteq \widehat{\rho}(x) \wedge \widehat{C}(\ell_2) \subseteq \widehat{C}(\ell)$
- [op] $(\widehat{C}, \widehat{\rho}) \models (t_1^{\ell_1} \ \mathbf{op} \ t_2^{\ell_2})^\ell$ iff $(\widehat{C}, \widehat{\rho}) \models t_1^{\ell_1} \wedge (\widehat{C}, \widehat{\rho}) \models t_2^{\ell_2}$
-

Inclusions

Note that the clauses contain a number of inclusions of the form

$$lhs \subseteq rhs$$

where:

- $lhs ::= \hat{C}(\ell) \mid \hat{\rho}(x), \{t\}$ where t is a function abstraction
- $rhs ::= \hat{C}(\ell) \mid \hat{\rho}(x)$

These express how functions may flow through the program, e.g.,

$$\hat{C}(\ell) \subseteq \hat{\rho}(x)$$

Observation 1

$[fn] (\hat{C}, \hat{\rho}) \models (fn\ x \Rightarrow e_0)^\ell$ iff $\{fn\ x \Rightarrow e_0\} \subseteq \hat{C}(\ell)$

$[app] (\hat{C}, \hat{\rho}) \models (t_1^{\ell_1} t_2^{\ell_2})^\ell$ iff $(\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2}$
 $(\forall (fn\ x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) :$
 $(\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge$
 $\hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge$
 $(\forall (fun\ f\ x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) :$
 $(\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge$
 $\hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge$
 $\{fun\ f\ x \Rightarrow e_0\} \subseteq \hat{\rho}(f))$

$[fn]$ (and $[fun]$) do **not** recursively analyze subexpressions

$[app]$ does the job (is responsible for analyzing function bodies)

the analysis is driven by reachability

In this way, we analyse the body of a function **only** if the function may be applied somewhere

Unreachable functions are not analysed

Observation 2

$$\begin{aligned}
 [app] \ (\hat{C}, \hat{\rho}) \models (t_1^{\ell_1} t_2^{\ell_2})^\ell \text{ iff } & \boxed{(\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2}} \\
 & (\forall (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) : \quad t_0^{\ell_0} \text{ is not a subexpression} \\
 & \quad \boxed{(\hat{C}, \hat{\rho}) \models t_0^{\ell_0}} \wedge \\
 & \quad \hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge \\
 & (\forall (\text{fun } f \ x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) : \\
 & \quad (\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge \\
 & \quad \hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge \\
 & \quad \{\text{fun } f \ x \Rightarrow e_0\} \subseteq \hat{\rho}(f))
 \end{aligned}$$

The relation $\models$ is not syntax-directed (due to the application clause)

So we cannot define $\models$ by structural induction on expressions

Instead, define $\models$ **coinductively**

Back to Example 2

$\text{loop} \triangleq (\text{let } g = (\text{fun } f \ x \Rightarrow (f^1 (\text{fn } y \Rightarrow y^2)^3)^4)^5$
 $\text{in } (g^6 (\text{fn } z \Rightarrow z^7)^8)^9)^{10}$

Here is an acceptable guess

$\hat{C}_{lp}(1) = \{f\}$	$\hat{C}_{lp}(6) = \{f\}$	$\hat{\rho}_{lp}(f) = \{f\}$
$\hat{C}_{lp}(2) = \emptyset$	$\hat{C}_{lp}(7) = \emptyset$	$\hat{\rho}_{lp}(g) = \{f\}$
$\hat{C}_{lp}(3) = \{\text{id}_y\}$	$\hat{C}_{lp}(8) = \{\text{id}_z\}$	$\hat{\rho}_{lp}(x) = \{\text{id}_y, \text{id}_z\}$
$\hat{C}_{lp}(4) = \emptyset$	$\hat{C}_{lp}(9) = \emptyset$	$\hat{\rho}_{lp}(y) = \emptyset$
$\hat{C}_{lp}(5) = \{f\}$	$\hat{C}_{lp}(10) = \emptyset$	$\hat{\rho}_{lp}(z) = \emptyset$

$f \triangleq \text{fun } f \ x \Rightarrow (f^1 (\text{fn } y \Rightarrow y^2)^3)^4$
 $\text{id}_y \triangleq \text{fn } y \Rightarrow y^2$
 $\text{id}_z \triangleq \text{fn } z \Rightarrow z^7$

- $\hat{\rho}_{lp}(g) = \{f\}$ reflects that g will be evaluated to a closure constructed from that abstraction
- $\hat{\rho}_{lp}(x) = \{\text{id}_y, \text{id}_z\}$ reflects that x will be bound to closures constructed from both abstractions in the course of evaluation
- $\hat{C}_{lp}(10) = \emptyset$ (no function value can be produced at this point) reflects that the evaluation of loop never terminates

Back to Example 2

$\hat{C}_{lp}(1) = \{f\}$	$\hat{C}_{lp}(6) = \{f\}$	$\hat{\rho}_{lp}(f) = \{f\}$
$\hat{C}_{lp}(2) = \emptyset$	$\hat{C}_{lp}(7) = \emptyset$	$\hat{\rho}_{lp}(g) = \{f\}$
$\hat{C}_{lp}(3) = \{id_y\}$	$\hat{C}_{lp}(8) = \{id_z\}$	$\hat{\rho}_{lp}(x) = \{id_y, id_z\}$
$\hat{C}_{lp}(4) = \emptyset$	$\hat{C}_{lp}(9) = \emptyset$	$\hat{\rho}_{lp}(y) = \emptyset$
$\hat{C}_{lp}(5) = \{f\}$	$\hat{C}_{lp}(10) = \emptyset$	$\hat{\rho}_{lp}(z) = \emptyset$

$f \triangleq \text{fun } f \ x \Rightarrow (f^1 (\text{fn } y \Rightarrow y^2)^3)^4$
 $id_y \triangleq \text{fn } y \Rightarrow y^2$
 $id_z \triangleq \text{fn } z \Rightarrow z^7$

from [fun]

$(\hat{C}_{lp}, \hat{\rho}_{lp}) \models \text{loop}$ iff $(\hat{C}_{lp}, \hat{\rho}_{lp}) \models f^5 \wedge ((\hat{C}_{lp}, \hat{\rho}_{lp}) \models g^6 (\text{fn } z \Rightarrow z^7)^8)$
 $\wedge \hat{C}_{lp}(5) \subseteq \hat{\rho}_{lp}(g) \wedge \hat{C}_{lp}(9) \subseteq \hat{C}_{lp}(10)$ ✓

using [app] and that $\hat{C}(6) = \{f\}$

immediate

$(\hat{C}_{lp}, \hat{\rho}_{lp}) \models g^6 \wedge (\hat{C}_{lp}, \hat{\rho}_{lp}) \models (\text{fn } z \Rightarrow z^7)^8 \wedge ((\hat{C}_{lp}, \hat{\rho}_{lp}) \models (f^1 (\text{fn } y \Rightarrow y^2)^3)^4)$
 $\wedge \hat{C}_{lp}(8) \subseteq \hat{\rho}_{lp}(x) \wedge \hat{C}_{lp}(4) \subseteq \hat{C}_{lp}(9) \wedge f \in \hat{\rho}_{lp}(f)$ ✓

immediate

Immediate

$(\hat{C}_{lp}, \hat{\rho}_{lp}) \models f^1 \wedge (\hat{C}_{lp}, \hat{\rho}_{lp}) \models (\text{fn } y \Rightarrow y^2)^3 \wedge ((\hat{C}_{lp}, \hat{\rho}_{lp}) \models (f^1 (\text{fn } y \Rightarrow y^2)^3)^4)$
 $\wedge \hat{C}_{lp}(3) \subseteq \hat{\rho}_{lp}(x) \wedge \hat{C}_{lp}(4) \subseteq \hat{C}_{lp}(4) \wedge f \in \hat{\rho}_{lp}(f)$ ✓

Circular dependency => coinduction: assume it holds and verify it is preserved

Coinductive nature of the CFA

The **specification** ($\models$) is **coinductive**

loose: includes many solutions

- It defines **all acceptable solutions**
- A solution is accepted by default: it is **rejected only if it violates a constraint**

Semantically this is a **greatest fixed point**

Computation

Among all acceptable solutions, we are interested in the smallest

We compute **the least solution satisfying the specification**

Algorithmically, this is a **least fixed point**

most precise safe approximation

The nature of this CFA

This is a 0-CFA analysis, characterised as:

```
let x = 0 in
  let x = 1 in
    x
```

Flow-sensitive $\rightarrow x = 1$
Flow-insensitive $\rightarrow x \in \{0, 1\}$

- **flow-insensitive**: it computes a single abstract value for each variable (or expression) that over-approximates all values it may take during execution, without distinguishing between different program points or execution orders
- **context-insensitive**: the same function is analyzed in the same way at all call sites

```
let f = fn x => x in
  let y = f 1 in
    let z = f 2 in
      ...
```

0-CFA analyzes f **once**
merging all calls

no information
about calling
context is used

Questions

Question 1

Let $e \triangleq ((\text{fn } y \Rightarrow (y^1 \ 2^2)^3)^4 (\text{fn } x \Rightarrow (x^5 * 3^6)^7)^8)^9$

Which are the labels?

Which is the flow from

1. the program point with label 5 to program point with label 7?
2. the program point with label 8 to variable y ?

1. No flow: constants are not tracked in 0-CFA
2. Flow from 8 to y : $\{\text{fn } x \Rightarrow (x^3 * 3^4)^5\}$ (via application)

Only functions flow in 0-CFA, not data

Question 2

Let $e \triangleq (\text{let } f = (\text{fn } x \Rightarrow x^1)^2 \text{ in } (f^3 4^4)^5)^6$

Which is the flow

- from the program point with label 2 to f ?
- from the program point with label 4 to x ?
- from the program point with label 5 to the program point with label 6?
- Flow from 2 to f : $\{\text{fn } x \Rightarrow x^1\}$
- No flow: 4 is a constant and data are not tracked
- No flow: since no value flows to x , nothing reaches label 6